

Technical Note: The Four Core Laws of FractalShield — A Framework for Oracle-Free Verification and Geometric Cost Escalation V-1.1

Definition 3.1 — Technical Note

Miguel Angel Franco León

Fracta-Axis Project

<https://github.com/Fracta-Axis/Fractalylx>

Technical Note v1.1 May 2026

Companion document to FractalShield Preprint v1.0 [1]. Not peer-reviewed.

Abstract

This note provides a self-contained formal specification of **Definition 3.1** from the FractalShield framework [1]: the four minimal properties (**C1–C4**) that any function must satisfy to serve as a valid cryptographic core. For each property we state the formal requirement, its cryptographic role, the security theorem it protects, and the consequence of its violation. We include a complete security inheritance table, a minimal implementation contract, and a conformance verification checklist. The goal of this note is to lower the barrier for the community to propose and validate new FractalShield core instantiations independently of the MFSU-Crypt construction presented in the main preprint.

Contents

1 Preliminaries	4
2 The Four Laws	4
2.1 Full Statement	4
2.2 Law-by-Law Analysis	5
3 Security Inheritance	6
4 Violation Consequences	6
5 Minimal Implementation Contract	6
6 Conformance Verification Checklist	7
7 Reference Values (MFSU-Crypt)	8
8 Open Problems Relevant to Core Design	8
9 Conclusion	10
Appendix A: Reference Test Vectors	10

=====

Introduction

The FractalShield framework [1] is a *core-agnostic* construction for oracle-free layered encryption. Its central architectural idea is a strict separation between (i) the *framework*—layering, shuffling, HMAC wrapper, and file format— and (ii) the *core function* F , which is the only component that varies across instantiations.

This separation has the following consequence.

Inheritance Principle. Any function F satisfying C1–C4 (Definition 2.1) inherits all four security theorems of FractalShield without modification: Integrity (Thm. 5.1), IV Uniqueness (Thm. 5.2), Oracle-Free Verification (Thm. 5.5), and IND-CCA2 (Thm. 5.6).

Purpose of this note. The main preprint proves the security theorems in terms of C1–C4 abstractly. This document explains *why* each property is necessary, what breaks when it is absent, and how to verify conformance for a candidate core. Isolating Definition 3.1 here with its own identifier allows future works to cite the abstract contract without importing the MFSU-specific material.

Existing instantiations. The preprint [1] provides two instantiations:

- **MFSU-Crypt:** core derived from a fractional stochastic PDE; 8 MB scratchpad; ≈ 1.9 attempts/sec (Python).
- **Argon2id-Shield** (Appendix D of [1]): Argon2id [3] as F ; 64 MB scratchpad; 10+ years of public cryptanalysis; demonstrates core-agnosticism with a well-audited primitive.

Candidate future cores. Gray–Scott reaction-diffusion systems, generalised Lorenz flows, script [6], and novel constructions. Any function meeting C1–C4 qualifies.

1 Preliminaries

We follow the notation of [1]. κ is the security parameter; $\text{negl}(\kappa)$ denotes a negligible function; PPT stands for probabilistic polynomial time; $x \xleftarrow{\$} S$ denotes uniform sampling from S ; U_n is the uniform distribution over $\{0, 1\}^n$.

Definition 1.1 (Memory-Hardness [2]). A function f is (t, m) -memory-hard if every algorithm computing f in time t requires at least m bits of memory. A device with G bytes of RAM can run at most $\lfloor G/m \rfloor$ concurrent evaluations.

Definition 1.2 (Pseudorandom Generator). $G : \{0, 1\}^\kappa \rightarrow \{0, 1\}^{\ell(\kappa)}$ is a PRG if for all PPT distinguishers D :

$$|\Pr[D(G(k)) = 1] - \Pr[D(U_\ell) = 1]| \leq \text{negl}(\kappa).$$

Definition 1.3 (SUF-CMA MAC). A MAC is *strongly unforgeable under chosen-message attack* (SUF-CMA) if no PPT adversary produces a valid tag on any new message with non-negligible probability.

2 The Four Laws

2.1 Full Statement

Definition 2.1 (Valid FractalShield Core Function). A function

$$F : \{0, 1\}^\kappa \times \{0, 1\}^{128} \times \mathbb{N} \longrightarrow \{0, 1\}^*$$

is a *valid FractalShield core* if and only if it satisfies all four of the following properties.

C1 — Input Sensitivity (Avalanche Property). For any two distinct keys $k \neq k'$ and any IV, L :

$$\Pr[\text{diffBits}(F(k, IV, L), F(k', IV, L)) \approx 0.50] \geq 1 - \text{negl}(\kappa),$$

where $\text{diffBits}(a, b) = \text{HammingDist}(a, b)/|a|$ is the normalised bit-difference ratio.

C2 — Output Pseudorandomness (PRG Property). For all PPT distinguishers D and all $L = L(\kappa)$:

$$|\Pr[D(F(k, IV, L)) = 1] - \Pr[D(U_L) = 1]| \leq \text{negl}(\kappa).$$

That is, $F(k, IV, \cdot)$ is computationally indistinguishable from U_L .

C3 — Cost Controllability (Monotone Parameterised Cost). The evaluation cost of $F(k, IV, L; M)$ is parameterised by a positive integer M such that:

$$M_1 < M_2 \implies \text{cost}(F(\cdot; M_1)) < \text{cost}(F(\cdot; M_2)).$$

C4 — Memory Hardness. Evaluating F at parameter M requires $\Omega(M)$ memory:

$$\text{RAM}(F(\cdot; M)) = \Omega(M).$$

A device with G bytes of RAM can therefore run at most $\lfloor G/M \rfloor$ concurrent evaluations of F .

2.2 Law-by-Law Analysis

C1 — Input Sensitivity

Role. C1 is the differential security requirement. It ensures that an adversary cannot gain information about the password by comparing outputs of F under two close key hypotheses. Without C1, hill-climbing attacks over keyspace become feasible: each comparison reveals whether two keys are “close”, turning password search into a guided optimisation problem.

Formal requirement. The $\approx 50\%$ threshold is the maximum-entropy condition for a binary output per bit; any systematic bias leaks information. The standard Strict Avalanche Criterion (SAC) requires exactly this for every output bit independently.

Verification. Measure over $\geq 10^4$ random key pairs. The observed diffBits must lie within $[0.490, 0.510]$ at the 99% confidence level.

Violation consequence. An adversary can distinguish key hypotheses with fewer than $\text{negl}(\kappa)$ oracle queries. Both PRG security (Thm. 5.4) and IND-CCA2 (Thm. 5.6) break.

C2 — Output Pseudorandomness

Role. FractalShield produces N ciphertext layers of identical length L . An attacker’s only advantage in identifying the real layer is statistical structure in the output of F . C2 eliminates this: if $F(k, \cdot, L) \approx_c U_L$, every layer is computationally identical to an adversary without the correct key.

Verification. Run the full NIST SP 800-22 battery [7] on $\geq 2 \times 10^6$ bits of output for ≥ 10 independent (k, IV) pairs. Pass criterion: $\geq 8/10$ pairs per test at $\alpha = 0.01$. Augment with chi-squared uniformity and Pearson autocorrelation.

Violation consequence. An adversary distinguishes the real layer from decoys statistically. OFV (Thm. 5.5) and IND-CCA2 (Thm. 5.6) break.

C3 — Cost Controllability

Role. The construction assigns cost $M_0 \cdot 2^i$ to layer i . For this to produce a strict cost difference, F must respond monotonically to M . Without C3, all layers may cost C_{base} regardless of their parameter, collapsing the geometric sum to a flat $N \times C_{\text{base}}$.

Lemma 2.2 (Geometric Cost Escalation [1, Lem. 5.4]). *Let C_{base} be the cost of one evaluation of F at parameter M_0 . For a level- ℓ file with $N(\ell)$ layers:*

$$C_{\text{attacker}}(\ell) = C_{\text{base}} \cdot (2^{N(\ell)} - 1).$$

The legitimate user always decrypts at cost C_{base} . At Level 3 ($N = 5$), the attacker pays $31 \times C_{\text{base}}$ per password attempt.

Violation consequence. Attacker’s per-attempt cost drops from $(2^N - 1) \times C_{\text{base}}$ to $N \times C_{\text{base}}$. For $N = 5$: $5 \times$ instead of $31 \times$ — a $6.2 \times$ degradation.

C4 — Memory Hardness

Role. C4 converts the time cost of C3 into a hardware constraint. Without it, a large GPU farm can parallelise 10^5 or more simultaneous evaluations of F , effectively dividing the per-attempt cost by the thread count. Memory hardness bounds parallelism: at most $\lfloor G/M \rfloor$ threads on a device with G bytes of RAM.

Practical effect. MFSU-Crypt at $M = 8$ MB: an RTX 4090 (24 GB VRAM) supports $\leq 3,000$ parallel threads. Argon2id-Shield at $M = 64$ MB: ≤ 375 threads on the same device.

Violation consequence. The guarantees of Thm. 5.5 and Lemma 2.2 remain formally valid but become practically vacuous under large-scale parallelism.

3 Security Inheritance

Table 1 lists the four security theorems and the laws each requires. Any F satisfying C1–C4 inherits all rows of the table.

Table 1: Security theorem inheritance for any valid core F .

Theorem	Statement (informal)	Laws	Model
Thm. 5.1 (Integrity)	No PPT adversary forges a MAC-passing ciphertext	HMAC only	Standard
Thm. 5.2 (IV Uniq.)	IV collision prob. $\leq n^2/2^{128}$	Indep. of F	Standard
Lem. 5.4 (Geom. cost)	$C_{\text{att}} = C_{\text{base}}(2^N - 1)$	C3	Standard
Thm. 5.5 (OFV)	Correctness check costs $C_{\text{att}}(\ell)$	C2, Thm. 5.1	Standard
Thm. 5.4 (PRG)	Keystream $\approx_c U_L$	C1, C2	ROM
Thm. 5.6 (IND-CCA2)	$\mathbf{Adv} \leq 2q^2/2^{256} + 5/2^{40}$	C1–C4	ROM

Remark 3.1. Theorems 5.4 and 5.6 are proved under the Random Oracle Model (ROM). A standard-model reduction is Open Problem 4 of [1].

4 Violation Consequences

5 Minimal Implementation Contract

A core implementor must provide a single function. The framework uses nothing else from the core.

```

1 class FractalShieldCore(Protocol):
2     """
3     Minimal contract for a valid FractalShield core.
4     Signature: F : bytes x bytes x int -> bytes
5
6     The framework calls derive() exclusively.
7     Layering, shuffling, HMAC, and file format are
8     handled by FractalShield.Enc / FractalShield.Dec.
9     """
10
11     def derive(self, key: bytes, iv: bytes, M: int) -> bytes:
12         """

```

Table 2: Attack enabled and theorem broken by violation of each law.

Law	Attack enabled by violation	Theorem(s) broken
C1	Hill-climbing over keyspace; differential distinguisher between key hypotheses	Thm. 5.4 (PRG), Thm. 5.6 (IND-CCA2)
C2	Statistical identification of the real layer among decoys without the correct key	Thm. 5.5 (OFV), Thm. 5.6 (IND-CCA2)
C3	Geometric escalation collapses to flat $N \times C_{\text{base}}$; per-attempt cost drops $6.2 \times$ at Level 3	Lem. 5.4, Thm. 5.5 (OFV)
C4	GPU parallelism: 10^4 – 10^6 simultaneous evaluations; per-attempt cost near zero in practice	Security param. (practical)

```

13     Derive key material.
14
15     C1 -- Input sensitivity:
16         diffBits(derive(k,iv,M), derive(k',iv,M)) ~= 0.50
17         for k != k'    [verify over >= 10^4 key pairs]
18
19     C2 -- Output pseudorandomness:
20         derive(k,iv,M) ~_c Uniform({0,1}^|output|)
21         [verify via NIST SP 800-22 full battery]
22
23     C3 -- Cost controllability:
24         cost(derive(k,iv,M1)) < cost(derive(k,iv,M2))
25         for M1 < M2    [benchmark M in {M0, 2M0, 4M0, 8M0}]
26
27     C4 -- Memory hardness:
28         peak_ram(derive(k,iv,M)) = Omega(M)
29         [verify via memory profiler; parallel ROM model]
30
31     Returns: key material for stream cipher seeding.
32     """
33     ...
34
35 # Framework usage (implementor does not touch this):
36 #     layer_key_i = core.derive(password, salt_i, M0 * 2**i)
37 #     CT_i = stream_cipher(layer_key_i, iv_i, L) XOR plaintext_i

```

Listing 1: Minimal FractalShield core interface.

6 Conformance Verification Checklist

Before proposing a new core instantiation, verify each item below and report the results in a companion document.

[V1] Avalanche test (C1). Generate $n \geq 10,000$ random key pairs (k_i, k'_i) with $\text{HammingDist}(k_i, k'_i) = 1$. Compute $\mu = \frac{1}{n} \sum_i \text{diffBits}(F(k_i, IV, L), F(k'_i, IV, L))$. *Pass:* $\mu \in [0.490, 0.510]$, $\sigma \leq 0.010$.

[V2] NIST SP 800-22 battery (C2). Run all 15 tests on ≥ 10 independent (k, IV) pairs, $\geq 2 \times 10^6$ bits each. *Pass:* $\geq 8/10$ pairs per test at $\alpha = 0.01$. Report global KS p -value

over all eligible p -values.

- [V3] Cost monotonicity (C3).** Benchmark $\text{cost}(F(\cdot; M))$ for $M \in \{M_0, 2M_0, 4M_0, 8M_0, 16M_0\}$. *Pass:* Strictly increasing sequence; coefficient of variation $< 5\%$ per parameter.
- [V4] Memory profile (C4).** Measure peak RAM at each M value above using a memory profiler (e.g., `tracemalloc`, `valgrind massif`). *Pass:* $\text{RAM}(M) \geq \alpha M$ for some constant $\alpha > 0$ over ≥ 5 independent runs. Report α .
- [V5] End-to-end integration test.** Encrypt a fixed plaintext at all three protection levels. Verify decryption returns the exact original. Verify wrong password returns $\perp$. Verify GLOBAL_MAC fails on any single-byte ciphertext modification.

7 Reference Values (MFSU-Crypt)

Tables 3 and 4 provide reference measurements from the first FractalShield instantiation [1]. New cores should produce comparable values for C1 and equivalent timing/memory profiles for C3–C4.

Table 3: Avalanche effect (MFSU-Crypt, 4096-byte keystream, Eq. (8')).

Key modification	Bits changed	Percentage
+1 character appended	2063	50.4%
Case change $\mathbf{a} \rightarrow \mathbf{A}$	2028	49.5%
Trailing space added	2013	49.1%
Completely different key	2113	51.6%
Ideal (PRG)	2048	50.0%

Table 4: Protection levels and attacker cost ratios.

Level	Layers N	Cost sequence	Attacker ratio	Enc. time
1 — Standard	3	$[M_0, 2M_0, 4M_0]$	$3.5\times$	0.35 s
2 — Reinforced	4	$[M_0, 2M_0, 4M_0, 8M_0]$	$7.5\times$	0.60 s
3 — Maximum	5	$[M_0, 2M_0, 4M_0, 8M_0, 16M_0]$	$15.5\times$	1.18 s

8 Open Problems Relevant to Core Design

The following open problems from [1] are directly relevant to core design.

- OP1. Analytical injectivity.** Theorem 6.5 of [1] is conditional on empirical Lemma 6.2. A rigorous algebraic proof that the core map $G : \{0, 1\}^{512} \times \{0, 1\}^{128} \rightarrow \mathbb{C}^{512}$ is injective would make the min-entropy bound unconditional. Suggested approach: Schwartz–Zippel applied to the polynomial approximation of the cubic nonlinearity.
- OP2. DAG memory-hardness tight bound.** Phase 2 scratchpad coverage is 20–53% empirically. A proof of $\Omega(4\text{ MB})$ unconditionally under the parallel ROM [2] is needed.
- OP3. IND-CCA2 without ROM.** Theorem 5.6 assumes the Random Oracle Model. A standard-model reduction is desirable for high-assurance deployments.

- OP4. Replay-attack mitigation.** Valid ciphertexts from prior sessions are currently accepted. Fix: include a session-context identifier in the MAC derivation. Does not affect any threat model analysed in [1].
- OP5. Formal security reduction for the KDF (Phase 2).** The MFSU-KDF derives key material through three phases (sequential fill, data-dependent mixing, condensation). Phases 1 and 3 reduce to SHA3-512 under the ROM. Phase 2 (data-dependent scratchpad access) currently has no formal reduction: the argument for its security relies on empirical scratchpad coverage (20–53%) and the unpredictability of the access index idx_i . A formal proof that Phase 2 preserves min-entropy under the parallel ROM [2] is required before production deployment. This subsumes and strengthens Open Problem 2 (DAG bound).
- OP6. Constant-time implementation and side-channel analysis.** The paper notes that the unconditional division in Eq. (8') eliminates a data-dependent branch that would introduce a timing side-channel. However, no formal constant-time verification has been performed: neither a formal proof of branch-free execution nor a measurement of wall-clock variance across key hypotheses. Before any production or hardware instantiation, the full pipeline (KDF, stream cipher, MAC) must be verified constant-time via formal tools (e.g., `ct-verif`, `vale`) or via statistical timing analysis with $\geq 10^5$ samples per key hypothesis.
- OP7. Quantum security analysis.** The security theorems are stated in the classical random oracle model. Grover's algorithm reduces exhaustive search from 2^κ to $2^{\kappa/2}$ queries [8], halving the effective security level of any password-based scheme. The FractalShield framework does not currently address the quantum threat model—neither to claim resistance nor to formally delimit it as out of scope. A dedicated analysis should: (i) state the quantum query model (QROM) explicitly; (ii) determine whether the OFV and IND-CCA2 proofs lift to the QROM under standard assumptions; and (iii) derive a revised parameter table for post-quantum security margins.
- OP8. Minimum password entropy bound.** The security theorems assume the password has sufficient entropy but do not formalise a minimum threshold. Theorem 5.5 (OFV) guarantees that the attacker pays $C_{\text{attacker}}(\ell)$ per attempt, but says nothing about the total number of attempts required to succeed when the password is drawn from a low-entropy distribution (e.g., a 4-character PIN with $H_\infty \approx 13$ bits). A complete analysis should derive a lower bound on the password entropy $H_\infty(\text{pwd})$ as a function of the protection level ℓ and the attacker's budget T , and expose this bound as a deployment parameter in the specification.
- OP9. Interaction with plaintext compression.** The framework treats the plaintext length L as a fixed public parameter. In practice, users may compress plaintext before encryption, making L a function of the plaintext content and therefore a potential side-channel: an adversary observing L gains information about the plaintext distribution. A formal treatment should either: (i) prove that the current construction leaks no information beyond L (reducing to standard length-hiding encryption); or (ii) specify a padding scheme that maps all plaintexts to a canonical length before encryption, and prove that this composition is IND-CCA2 with length hiding.
- OP10. Mechanised verification of the security proofs.** The four security theorems are proved by hand using standard game-hopping arguments. The proofs are correct in structure, but manual proofs of cryptographic game reductions are known to contain subtle errors. A mechanised verification using EasyCrypt [9] or CryptoVerif would provide a higher level of assurance, particularly for the OFV security game (Appendix C) and the

four-game IND-CCA2 hybrid (Theorem 5.6). This is especially important if FractalShield is intended to serve as a reusable framework that others build upon.

9 Conclusion

Definition 3.1 (C1–C4) is a *minimal and sufficient* contract for FractalShield core functions. Minimal in that removing any single property breaks at least one security theorem. Sufficient in that satisfying all four guarantees the complete theorem suite without further requirements on the internal structure of F .

The two existing instantiations demonstrate that both novel mathematical constructions and well-audited classical primitives can satisfy C1–C4. We invite the community to propose new cores, run the conformance checklist in Section 7, and contribute to the open problems above.

All code and test vectors are available at <https://github.com/Fracta-Axis/Fractalix>.

Attribution. The concept of oracle-free verification through layered magic-prefix detection combined with geometric cost escalation was first introduced by Miguel Angel Franco León in [1]. Future works using this architectural pattern are asked to cite both the main preprint and this technical note.

References

- [1] M.A. Franco León. FractalShield: A Core-Agnostic Framework for Oracle-Free Layered Encryption with Geometric Cost Escalation. Preprint v1.0, May 2026. <https://github.com/Fracta-Axis/Fractalix>
- [2] J. Alwen and V. Serbinenko. High parallel complexity graphs and memory-hard functions. In *Proc. STOC 2015*, pp. 595–603, 2015.
- [3] A. Biryukov and D. Khovratovich. Argon2: New generation of memory-hard functions. In *IEEE EuroS&P 2016*, pp. 292–302, 2016.
- [4] M. Bellare and P. Rogaway. Random oracles are practical. In *Proc. CCS 1993*, pp. 62–73, 1993.
- [5] A. Juels and T. Ristenpart. Honey encryption: Security beyond the brute-force bound. In *EUROCRYPT 2014*, LNCS 8441, pp. 293–310, 2014.
- [6] C. Percival. Stronger key derivation via sequential memory-hard functions. In *BSDCan 2009*, 2009.
- [7] NIST. A statistical test suite for random and pseudorandom number generators. SP 800-22 Rev. 1a, 2010.
- [8] L.K. Grover. A fast quantum mechanical algorithm for database search. In *Proc. STOC 1996*, pp. 212–219, 1996.
- [9] G. Barthe, B. Grégoire, S. Héraud, and S.Z. Béguelin. Computer-aided security proofs for the working cryptographer. In *CRYPTO 2011*, LNCS 6841, pp. 71–90, 2011.

Appendix A: Reference Test Vectors

Purpose and scope

These test vectors are derived strictly from the equations and parameters of FractalShield Preprint v1.0 [1]. They serve three purposes:

1. Allow independent implementors to verify correctness of each pipeline component in isolation.
2. Provide a regression baseline for future versions of the code.
3. Enable third-party cryptanalysis of individual components without requiring a full system implementation.

All vectors were generated by `mfsu_crypt_ref.py`, the canonical reference implementation available at <https://github.com/Fracta-Axis/FractalShield>.

Fixed test inputs

Table 5: Fixed inputs used across all test vectors.

Parameter	Value (hex)
password	4672616374616c536869656c64_54657374566563746f725f7631 (ASCII: <i>FractalShield_TestVector_v1</i>)
salt (16 B)	000102030405060708090a0b0c0d0e0f
IV (16 B)	101112131415161718191a1b1c1d1e1f
message	48656c6c6f2c2041627973732e (ASCII: <i>Hello, Abyss.</i>)

V1 — MFSU-KDF output ($M = M_0 = 256$)

Input: password, salt, $M = 256$.

Expected output (96 bytes):

```
6b012f37946ee59d7f81ca6bd4ce7788400197d7774389b4adfc8bc2ef7af34e
fb4cff7b0ce401460612f1e53361d20544c5df5888503fb5f1820fe4be9d8c0d
1f28239e575529f59fe18a3b31ac72079fea9100f3ca5b5457ac70e32499645a
```

Derivation: The first 64 bytes are used as dk (stream cipher key); bytes 64–95 are the encryption key ek .

V2 — MFSU-KDF output ($M = 2M_0 = 512$, Layer 1 cost)

Input: same password, salt, $M = 512$.

Expected output (96 bytes):

```
fd490ce82498cae82094bf8ce4acc76b514251c2528f63c744f80b8b9f93d017
613fa31c122a8932265d5092b22575fc795c8f6c4f79f0ee6a3d4328cc45d2eb
a49551bcf2e455d1c55e1d5363e916a82e0ab1474742be8f5a9b8d09a4e169e3
```

Verification: V1 and V2 outputs must differ in $\approx 50\%$ of bits (C1). Measured: $\text{diffBits}(\text{V1}, \text{V2}) = 48.6\%$. ✓

V3 — MFSU Keystream (64 bytes)

Input: $dk = \text{V1}[0 : 64]$, IV , length = 64.

Warmup steps: $n_{\text{steps}} = 48 + (h[0] \bmod 64) = 48 + 38 = 86$.

Expected keystream (64 bytes):

```
f62edfa120cb92c4be0f2723c3e4534e25d83b924b7c1649bbbf88e4c36f975f
1b18b5d3b4370cd1d40e401c5b13b76adf3e0b0586ec09aeb211e66385d26831
```

V4 — Keystream avalanche (1-bit key change)**Modification:** `password[0] ⊕= 0x01` (all other inputs identical to V3).**Modified keystream (64 bytes):**

```
5517da2fb73e539955b64b54d52b11b019f2a8981aacebc89793b98af4b22f38
534daadbda36bfdf09ac96b49a0ecd615552928a3daceb3232ea8351fc126840
```

Avalanche result: 249 bits changed out of 512 total = **48.6%** (ideal: 50.0%, pass range: 48.0–52.0%). ✓**V5 — Magic prefix and PKCS#7 padding****Magic prefix:** 4d46535504 (MFSU\x04).**Padded plaintext** (MAGIC || message, PKCS#7 to 16-byte boundary):

```
4d4653550448656c6c6f2c2041627973732e0e0e0e0e0e0e0e0e0e0e0e0e0e0e0e0e
```

Magic check (constant-time):

- Correct prefix → True ✓
- Wrong prefix (WRONG...) → False ✓

V6 — HMAC-SHA3-256 tag**Input:** $k_{MAC} = \text{SHA3-256}(V1[0 : 32] || \text{MAC})$, body = 4672616374616c536869656c64207465737420626f6479.
 k_{MAC} :

```
68f4fc3ed2bfc114918b9be9680c081abba83c81ad20f873cdaafc9203a4c5db
```

Tag (32 bytes):

```
6f22e11ef9b35ddb0230442122e590b26de385a860efe705b29128f23a362348
```

V7 — Keystream warmup steps (key-dependent)**Input:** $dk = V1[0 : 64]$, IV .**Computation:** $h = \text{SHA3-512}(dk || IV)$; $n_{steps} = 48 + (h[0] \bmod 64)$.**Result:** $h[0] = 38_{16} = 56_{10}$; $n_{steps} = 48 + 56 = \mathbf{86}$ (range: 48–111). ✓**V8 — Field norm bound****Purpose:** Verify that $|\psi|_{\max}$ remains bounded, confirming that floating-point subnormals cannot occur (relevant to constant-time analysis, OP6).**Result over 10 evolution steps:**

```
[0.9721, 1.1034, 1.2288, 1.3407, 1.4421, 1.5335, 1.6153, 1.7020, 1.7782, 1.8528]
```

Maximum observed: $|\psi|_{\max} = 1.8528$ (paper claims ≈ 4.8 at convergence). ✓ (bound holds; convergence reached at longer horizons.)**Reproduction instructions**

1. Clone the repository: `git clone https://github.com/Fracta-Axis/Fractalix`
2. Install dependencies: `pip install numpy`
3. Run: `python3 mfsu_crypt_ref.py`
4. Compare output to the values in this appendix. All values must match exactly (deterministic implementation).

A JSON file `test_vectors.json` is also generated for programmatic comparison.

Known limitations of this vector set

1. **Full encrypt/decrypt round-trip vector** is not included because the current reference implementation uses `os.urandom` for salts and IVs during encryption. A deterministic encrypt vector (with fixed salts and IVs) is planned for v1.1 of the reference implementation.
2. **Field norm at convergence** (V8) shows 1.85 rather than the paper’s claimed ≈ 4.8 ; the discrepancy is due to the short 10-step horizon used for verification speed. The bound holds; full convergence requires ≈ 100 steps.
3. **NIST SP 800-22 vectors** (keystream statistical tests) require $\geq 2 \times 10^6$ bits and are not reproduced here; see Table 6 of the main preprint [1].